Listing 5.4 `parallel_functions.py` (`run_conversation`, `request`)

```
user = """Can you please make recommendations for the following:
1. Time travel movies
2. Recipes
3. Gifts"""     #1
messages = [{"role": "user", "content": user}]     #2
tools = [     #3
    {
        "type": "function",
        "function": {
            "name": "recommend",
            "description":
                "Provide a recommendation for any topic.",
            "parameters": {
                "type": "object",
                "properties": {
                    "topic": {
                        "type": "string",
                        "description":
                              "The topic, … recommendation for.",
                        },
                        "rating": {
                            "type": "string",
                            "description": "The rating … was given.",
                            "enum": ["good", "bad", "terrible"]
                            },
                        },
                "required": ["topic"],
                },
            },
        }
    ]
```

#1 The user message asks for three recommendations.
#2 Note that there is no system message.
#3 Adds the function definition to the tools part of the request

Listing 5.5 shows the request being made, which we've covered before, but there are a few things to note. This call uses a lower model such as GPT-3.5 because delegating functions is a more straightforward task and can be done using older, cheaper, less sophisticated language models.

Listing 5.5 `parallel_functions.py` (`run_conversation`, API call)

```
response = client.chat.completions.create(
    model="gpt-3.5-turbo-1106",     #1
    messages=messages,     #2
    tools=tools,     #2
    tool_choice="auto",   #3
)
response_message = response.choices[0].message     #4
```

#1 LLMs that delegate to functions can be simpler models.
#2 Adds the messages and tools definitions
#3 auto is the default.
#4 The returned message from the LLM